

PDS-CREACIONALES: ABSTRACT FACTORY



ING. ZOILA RUIZ, PHD

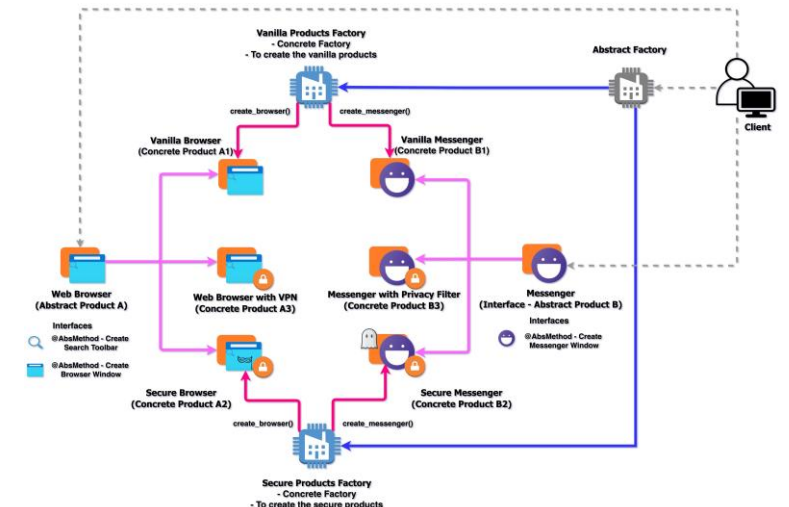
ABSTRACT FACTORY

Es un patrón de diseño creacional, conocido como fábrica de fábricas, que permite crear objetos de una misma familia de objetos relacionados o dependientes, sin especificar sus clases concretas.

En este patrón tenemos una fábrica concreta para una categoría de productos concretos relacionados, es decir, tenemos varios métodos create, cada uno devuelve un producto concreto.

La diferencia principal es que Factory Method se encarga de la creación de objetos únicos; es decir, se crea un objeto de varias posibles clases candidatas.

En el caso de Abstract Factory, lo que queremos es crear familias de objetos que mantengan una relación entre ellos.



ABSTRACT FACTORY

Abstrac Factory

- ❖ Crea grupos o familias que tienen una relación de herencia o dependencia entre ellos.
- ❖ Permite que la creación de objetos mantenga una lógica y coherencia, manteniendo un alto nivel de abstracción y bajo acoplamiento a través del polimorfismo.
- ❖ Se abstrae la forma de creación de objetos sin conocer detalles de creación o inicialización y del tipo concreto de objeto creado en ejecución.
- ❖ Proporciona un método de creación por cada tipo de objetos que pertenece a la familia o grupo a crear.

Es especialmente útil cuando el sistema debe ser independiente de cómo se crean, componen y representan sus objetos.

ABSTRACT FACTORY

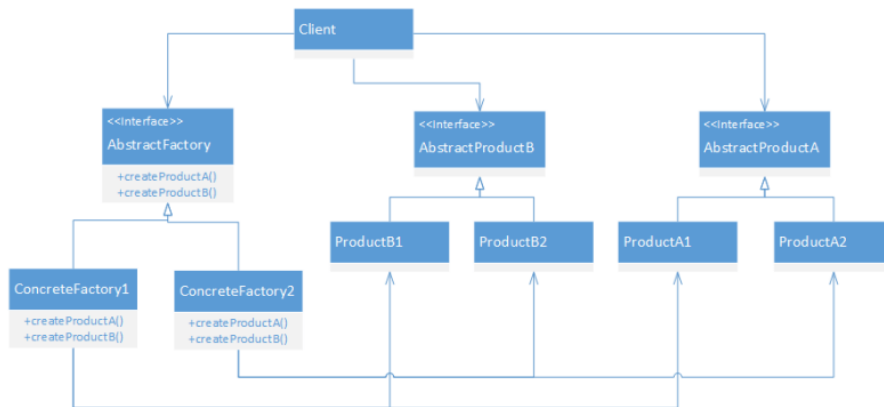
Contexto: Estás desarrollando un sistema que debe funcionar con diferentes familias de objetos relacionados (por ejemplo, interfaz de usuario para Windows, macOS, etc.).

Problema: ¿Cómo crear familias de objetos relacionados sin acoplar el código a sus clases concretas?

Solución: Crear una interfaz para fabricar objetos de una familia y subclases concretas que implementan esa interfaz para cada familia.

CONSECUENCIAS POSITIVAS	CONSECUENCIAS NEGATIVAS
Asegura consistencia entre objetos relacionados	Complejidad adicional
Aislamiento de clases concretos	Difícil de extender para nuevas familias
Facilita la interoperabilidad	
Facilita el mantenimiento y escalabilidad	

ABSTRACT FACTORY



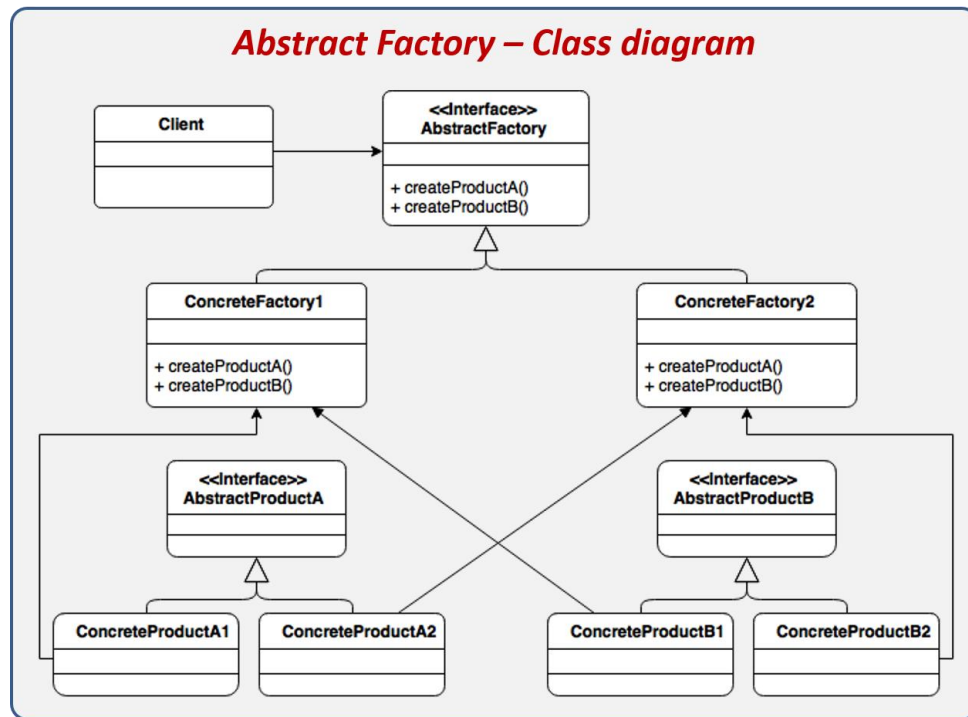
Se debe declarar de forma explicita interfaces/clases abs para cada producto diferente de la familia de productos

Cada variante de productos individuales debe extender o implementar la clase o interfaz

Debemos declarar la AbstractFactory como una interfaz con una lista de métodos de creación para todos los productos que son parte de la familia de productos.

Para cada variante de una familia de productos, creamos una clase de fábrica independiente basada en la interfaz AbstractFactory. Estas variantes devolverán productos de un tipo particular.

ABSTRACT FACTORY



La arquitectura del patrón tiene los siguientes elementos:

AbstractFactory: Define una interfaz para crear objetos abstractos de una familia

ConcreteFactory: Implementa la interfaz de la fábrica abstracta, y devuelve objetos concretos específicos de una variante.

AbstractProduct: Declara una interfaz común para una clase de productos

ConcreteProduct: Implementa la interfaz del producto y define el producto específico que pertenece a una variante concreta.

Client: Utiliza solo las interfaces abstractas para crear objetos. Es independiente de las clases concretas que se instancian.

ABSTRACT FACTORY

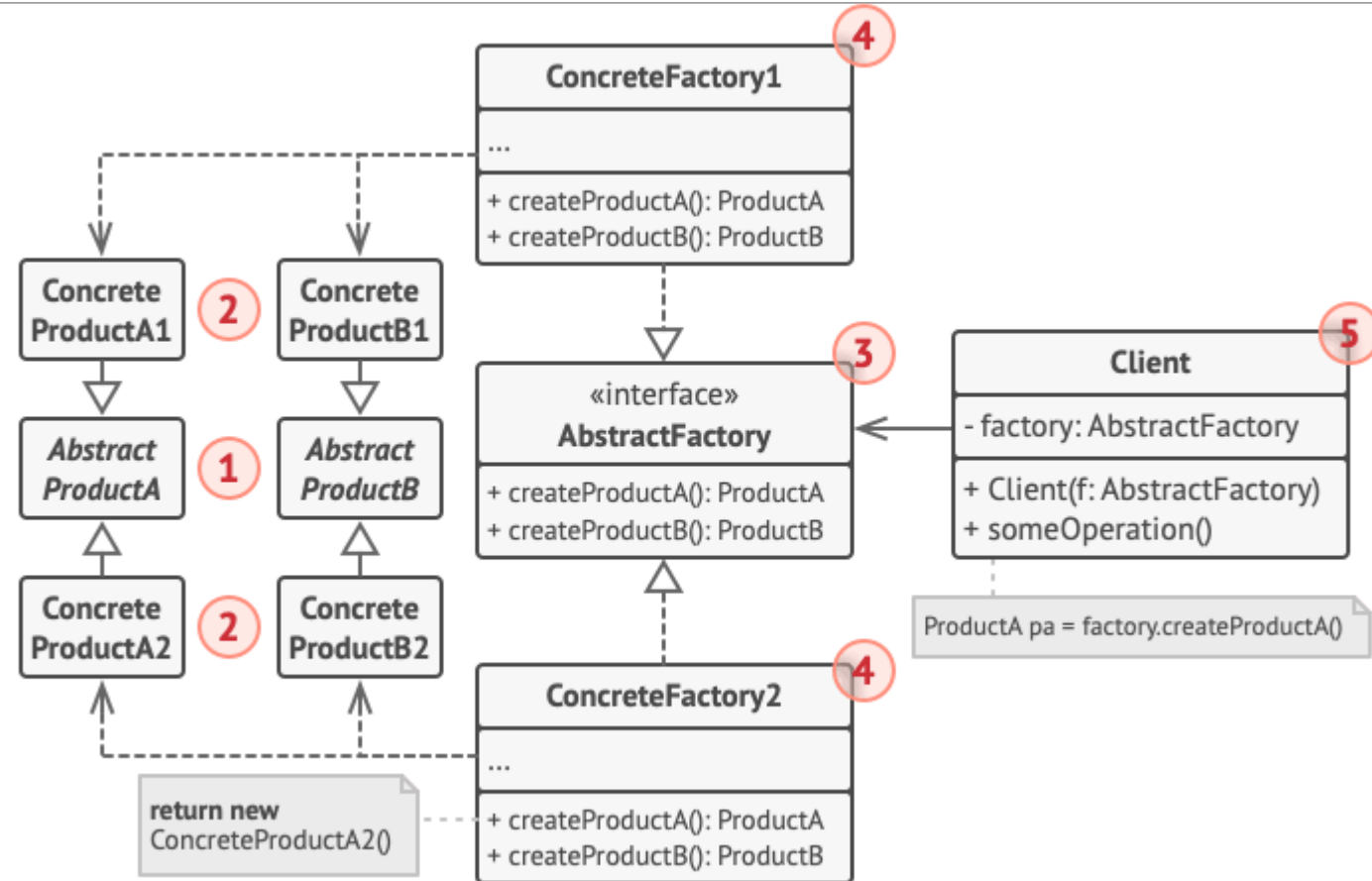
Usos:

Cuando un sistema debe ser independiente de como sus objetos son creados

Un sistema debe ser configurado con una cierta familia de productos

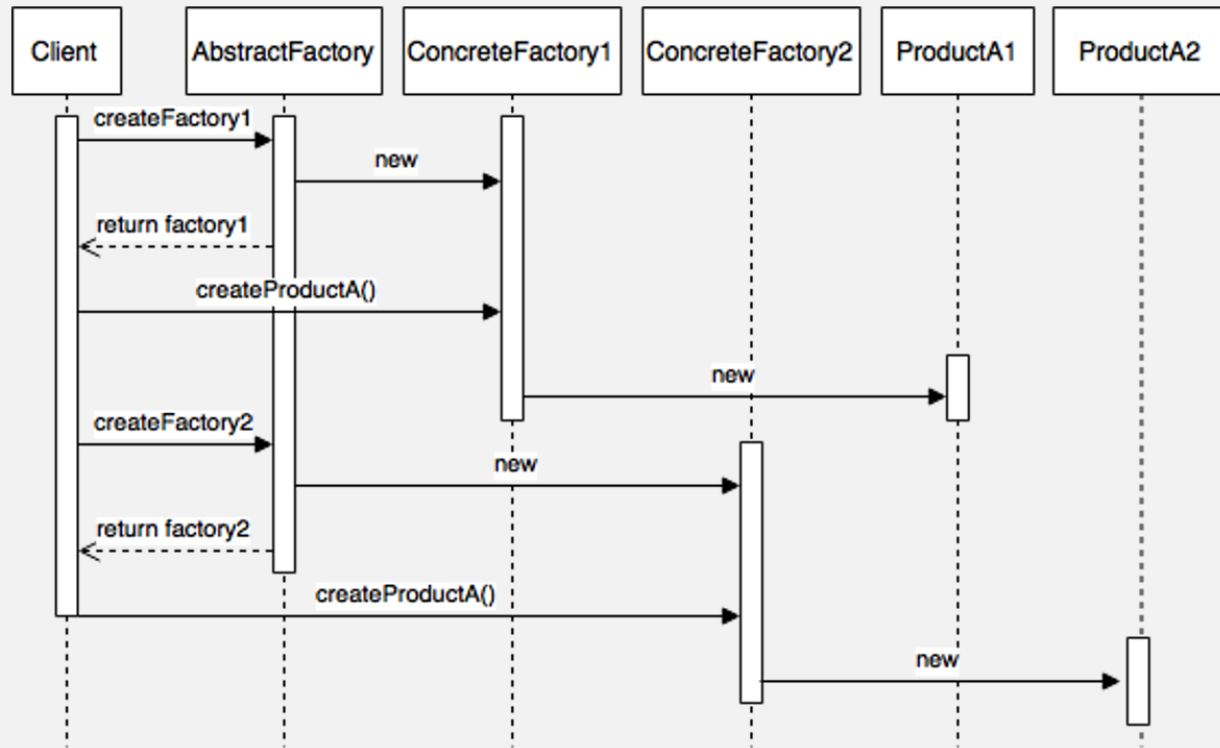
Se necesita reforzar la noción de dependencia mutua entre ciertos objetos.

ABSTRACT FACTORY



ABSTRACT FACTORY

Abstract Factory – Diagram of sequence



- ❖ Permite que el código cliente use objetos sin conocer sus clases concretas.
- ❖ Garantiza la compatibilidad entre objetos.
- ❖ Permite introducir nuevas variantes de productos sin cambiar el código cliente.
- ❖ Promueve una arquitectura más modular, separando claramente la creación de objetos de su uso